GU-TECH MID EXAM 2025

PF LAB

Date:	**Name:**
Program:	**Course: DB Lab**
Total Marks: 30 Marks	**Instructor's Name:**

Question 1:

Database Schema

`Students`

- `StudentID` (INTEGER)
- `FirstName` (VARCHAR)
- `LastName` (VARCHAR)
- `Major` (VARCHAR)
- `GPA` (DECIMAL)

`Courses`

- `CourseID` (VARCHAR) - *Example: "CS101"*
- `CourseName` (VARCHAR)
- `Department` (VARCHAR)
- `Credits` (INTEGER)

`Enrollments`

- `StudentID` (INTEGER) - *This links to the Students table*
- `CourseID` (VARCHAR) - *This links to the Courses table*
- `Semester` (VARCHAR) - *Example: "Fall 2025"*
- `Grade` (DECIMAL) - *Example: 3.7*

Section A: Creating Tables & Inserting Data
Make sure to enter sufficient and diverse data values. Include as many realistic combinations as possible. Data entry carries marks, so your dataset will be evaluated for completeness and variety.

1. `CREATE TABLE` **(Students):** Write the full SQL command to create the `Students` table.
2. `CREATE TABLE` **(Courses):** Write the full SQL command to create the `Courses` table.
3. `CREATE TABLE` **(Enrollments):** Write the full SQL command to create the `Enrollments` table.

4. `INSERT` **Multiple Students:** Write the SQL command to insert the following **three** students into the `Students` table using a **single** `INSERT` statement.
    - (101, 'Sarah', 'Connor', 'Computer Science', 3.8)
    - (102, 'John', 'Smith', 'History', 3.2)
    - (103, 'Mary', 'Jane', 'Biology', 3.9)
5. `INSERT` **Multiple Courses:** Write the SQL command to insert the following **two** courses into the `Courses` table using a **single** `INSERT` statement.
    - ('CS101', 'Intro to Programming', 'Computer Science', 3)
    - ('HIST150', 'World History', 'History', 3)
6. `INSERT` **Multiple Enrollments:** Write the SQL command to insert the following **two** enrollments into the `Enrollments` table.
    - Student 101 enrolled in 'CS101' in 'Fall 2025' and got a 3.7.
    - Student 102 enrolled in 'HIST150' in 'Fall 2025' and got a 3.4.

Section B: Querying Data (Joins & Aggregation)

For this section, assume the tables are now fully populated with data.

7. Write a query to find the `FirstName` and `LastName` of all students enrolled in the course "CS101". .
8. Write a query to list the `FirstName` of students and the `CourseName` of every course they are enrolled in. The result should have two columns: `FirstName` and `CourseName`.
9. Write a query to find the names (`CourseName`) of all courses taken by the student named "John Smith".
10. Write a query to find the **average GPA** of all students, by their `Major`.
11. Write a query to count the number of students enrolled in each course. The result should show the `CourseID` and the `StudentCount` for each course.
12. Building on your answer from question 11, modify the query to only show courses that have **more than 30** students.
13. Write a query to find the average `Grade` for the course "CS101" during the "Fall 2025" semester.
14. Write a query to find the `StudentID` of any student who is enrolled in **more than 4** courses.

15. For each **department**, calculate:
    - total number of students enrolled
    - average grade across all enrollments
    - number of unique courses offered
    - Show only those departments whose **average grade is higher than 3.2**, and sort them by highest average grade first.

16. List students who are taking **more than 12 total credits** *in a single semester*.
    Show:

- Student full name
- Semester
- Total credits taken

Question 2:

You are now working with a database for a small electronics retailer.

Database Schema

1. **`Products`**
    - `ProductID` (INTEGER)
    - `ProductName` (VARCHAR)
    - `Category` (VARCHAR) - *Example: "Laptops", "Smartphones", "Accessories"*
    - `SupplierID` (INTEGER)
2. **`Suppliers`**
    - `SupplierID` (INTEGER)
    - `SupplierName` (VARCHAR)
    - `City` (VARCHAR)
3. **`Sales`**
    - `SaleID` (INTEGER)
    - `ProductID` (INTEGER) - *Links to Products table*
    - `QuantitySold` (INTEGER)
    - `SalePrice` (DECIMAL) - *Price per unit at the time of sale*

17**.** The company tracks sales performance by supplier. Write a single SQL query to identify all **Suppliers** who supply **more than 5 products** AND whose **total revenue from sales is less than $5,000**.
The query should return the SupplierName.

18. List the `SupplierName` and the count of how many `Products` they supply, including any **Suppliers who supply zero products** (display `0`).

19. Show pairs of products that belong to the same category and share the same supplier.

20. Show all products that were sold at more than one distinct price.

21. Category with the highest total quantity sold.

Question 3:

22. **Design an Inventory Tracking Table for a warehouse management system** with the following requirements:

1. Each record tracks the stock of a specific product in a specific warehouse.
2. Include the following fields:
    - InventoryID: Unique identifier for each inventory record.
    - ProductID: Reference to a product in the Products table.
    - WarehouseID: Reference to a warehouse in the Warehouses table.
    - Quantity: Amount of product in stock. Should allow fractional units.
    - LastRestockDate: Date when the product was last restocked.
    - ExpirationDate: Optional field for the product's expiration date, if applicable.
    - Notes: Optional text notes about the inventory.
3. Requirements:
    - Use appropriate data types for IDs, quantities, and dates.
    - Show logical references to Products and Warehouses tables.

Optional Challenge:

- Choose data types that optimize storage and precision (e.g., for quantities and monetary values if used in future calculations).

Give a complete query to create this table above.

23. **Design a Seasonal Promotions Table for an e-commerce platform** with the following requirements:

1. Promotions can apply to specific products, entire categories, or both.
2. Include the following fields:
    - PromoID
    - PromoName
    - ProductID
    - Category
    - DiscountAmount
    - StartDate
    - EndDate
    - ActiveFlag
3. Requirements:
    - Use appropriate data types for each field.
    - Design the table to allow flexibility for promotions that may apply to products, categories, or both.

Give a complete query to create this above table above..